

ParentConnect Project Report

Complete Project Analysis

Date: February 19, 2026

Codebase: ParentConnect

Generated by: Antigravity AI

Chapter 1 : Project Overview

ParentConnect is a web application designed to bridge the gap between schools and parents, particularly those who may not be fluent in English. It provides a real-time dashboard of student performance, attendance, and notifications, accessible in multiple Indian languages (English, Telugu, Hindi, Tamil).

The core innovation lies in its AI capabilities during the project:

- Performance Prediction: Uses machine learning to predict future marks based on historical trends.
- Multilingual Chatbot: An intelligent assistant that answers parent queries in their native language.

Chapter 2 : Technology Stack

1. Backend:

- Python 3.x with Flask Web Server
- AI/ML: scikit-learn (Linear Regression), numpy
- NLP: deep_translator (Google Translate), gTTS (Text-to-Speech)
- Utilities: flask-cors

2. Frontend:

- HTML5, CSS3, Vanilla JavaScript (ES6+)
- Visualization: Chart.js
- Voice: Web Speech API for speech recognition

3. Data Storage:

- In-memory Python dictionaries simulating a database structure for Parents, Students, Marks, and Notifications.

Chapter 3 : Key Features

A. Multilingual Support

- Supports English, Telugu, Hindi, and Tamil.
- Static UI translations + Dynamic content translation via Google Translate API.

B. AI-Powered Dashboard

- Marks Visualization with color-coded indicators.
- Trend Analysis using Line Charts.
- Prediction Engine forecasting next semester marks with confidence scores.

C. Chat Assistant

- Context-aware chatbot (knows student's specific marks).

ParentConnect Project Report

- Voice-enabled (Speech-to-Text input, Text-to-Speech output).
- Intent recognition for queries like 'attendance', 'marks', 'prediction'.

Chapter 4 : File Structure

- d:/parentconnect/
 - app.py: Main Flask application entry point.
 - models.py: Machine Learning logic (PredictiveModel class).
 - chatbot.py: Chatbot logic (Intent matching).
 - requirements.txt: Python dependencies.
 - static/js/app.js: Frontend logic.
 - templates/: HTML templates for Login and Dashboard.

Chapter 5 : API Endpoints

- / (GET): Login page
- /dashboard (GET): Dashboard page
- /api/login (POST): Authenticate user
- /api/dashboard (GET): Get translated student data
- /api/translations/<lang> (GET): Get UI strings
- /api/chat (POST): Chatbot interaction
- /api/predict (POST): Custom prediction
- /api/tts (POST): Text-to-Speech